

Módulo 19: Seguridad y mantenimiento

Contenidos clave

Gestión de secretos y configuración: Un secreto es información que no debería estar escrita directamente en el código: contraseñas, API keys, tokens JWT, credenciales de bases de datos y claves privadas.

- Pydantic Settings: permite definir la configuración de la aplicación mediante un modelo tipado.

Auditoría de dependencias: La aplicación depende de paquetes externos que pueden descubrir vulnerabilidades por lo que se deben revisar las dependencias periódicamente:

- **pip-audit:** analiza las dependencias Python instaladas y busca vulnerabilidades conocidas, así se debe investigar una versión corregida.
- **safety:** también permite revisar dependencias buscando vulnerabilidades conocidas, su objetivo es similar al de pip-audit, ambas detectan vulnerabilidades conocidas dentro del alcance de sus fuentes/reglas.

Al encontrar una vulnerabilidad:

- Primero investigar qué versión corrige el problema
- Después actualizar
- Probar/Auditar, ya que actualizar una dependencia puede introducir incompatibilidades.

Actualizaciones (PEP 440) y compatibilidad:

- PEP 440: define como se representan y comparan las versiones de paquetes de Python; versiones preliminares, post releases, dev releases, etc..

La compatibilidad es importante porque si actualizamos automáticamente a una versión que introduce cambios incompatibles, la aplicación podría dejar de funcionar, por eso el flujo debe ser: Actualizar -> Ejecutar tests -> Auditar -> Revisar cambios -> Aceptar.

Hardening de contenedores y del runtime: Hardening significa reducir la superficie de ataque y limitar lo que puede hacer un sistema.

- Principio de mínimo privilegio: Un componente debe tener únicamente los permisos que necesita para realizar su función.
- Imagen mínima: En lugar de una imagen innecesariamente grande, cuando sea compatible con la aplicación.
- No incluir secretos
- Ejecutar como usuario no root
- Permisos mínimos

Hardening del runtime. No sólo importa cómo se construye el Dockerfile, también importa cómo se ejecuta el contenedor, se puede limitar: CPU, memoria, permisos, capacidades de Linux, acceso a filesystem y privilegios del proceso; si el proceso no tiene privilegios elevados, el impacto potencial puede reducirse.

Objetivos

- Proteger secretos y reducir superficie de riesgo.

Se integró pydantic-settings, la configuración se carga desde variables de entorno y .env:

```
class Settings(BaseSettings):
    model_config = SettingsConfigDict(
        env_prefix="ORDERS_",
        env_file=".env",
        env_file_encoding="utf-8",
        extra="ignore",
    )
```

Se utilizó .env para valores locales, mientras que .env.example documenta las variables necesarias sin exponer secretos reales. Se detectaron 2 vulnerabilidades, y se tuvo que migrar de python-jose a PyJWT para eliminar la dependencia vulnerable ecdsa.

El contenedor Docker ejecuta la aplicación con un usuario no root:

```
PS C:\Users\Quiryat\Documents\Python\python-mastery-capstone> docker run --rm orders-api:local id
uid=1000(appuser) gid=1000(appuser) groups=1000(appuser)
PS C:\Users\Quiryat\Documents\Python\python-mastery-capstone>
```

Reduciendo los permisos disponibles y la superficie de ataque.

- Mantener dependencias seguras y actualizadas.

Se integró pip-audit donde inicialmente se detectaron vulnerabilidades en black y ecdsa, se resolvieron hasta que la auditoría final mostró:

```
PS C:\Users\Quiryat\Documents\Python\python-mastery-capstone> poetry run pip-audit --local

No known vulnerabilities found
Name                               Skip Reason
-----
-----
```

Laboratorio

- Integrar pydantic-settings:

__init__.py	20
settings.py	21
Evidencias	22

Se creó una clase Settings para centralizar:

- URL de la API.
- Timeout
- Secreto JWT
- Algoritmo JWT
- Entorno de ejecución

Las variables se validan, incluyendo la longitud mínima del secreto JWT.

- Ejecutar pip-audit/safety y resolver hallazgos:

Se ejecutó pip-audit y se encontraron dos vulnerabilidades:

```
(python-mastery-capstone-py3.14) PS C:\Users\Quiryat\Documents\Python\python-mastery-capstone> poetry run pip-audit
Found 6 known vulnerabilities in 2 packages
Name Version ID Fix Versions
-----
black 24.2.0 PYSEC-2024-48 24.3.0
black 24.2.0 PYSEC-2024-48 24.3.0
black 24.2.0 PYSEC-2026-2121 26.3.1
black 24.2.0 PYSEC-2026-2120 26.3.0
ecdsa 0.19.2 PYSEC-2026-1325
ecdsa 0.19.2 PYSEC-2026-1325

Name Skip Reason
-----
python-mastery-capstone Dependency not found on PyPI and could not be audited: python-mastery-capstone (0.1.0)
```

- Black se actualizó a una versión corregida
- ecdsa se eliminó al reemplazar python-jose por PyJWT
- Se regeneraron poetry.lock y el entorno virtual

La auditoría final mostró:

```
PS C:\Users\Quiryat\Documents\Python\python-mastery-capstone> poetry run pip-audit --local
No known vulnerabilities found
Name Skip Reason
-----
python-mastery-capstone Dependency not found on PyPI and could not be audited: python-mastery-capstone (0.1.0)
```

Safety no se integró porque presentaba conflicto de versiones con Typer.

- Docker sin root y permisos mínimos:

Se verificó que la imagen Docker ejecuta la aplicación con:

```
PS C:\Users\Quiryat\Documents\Python\python-mastery-capstone> docker run --rm orders-api:local id
uid=1000(appuser) gid=1000(appuser) groups=1000(appuser)
PS C:\Users\Quiryat\Documents\Python\python-mastery-capstone>
```

Y no con root.

También se validó que FastAPI funcionara correctamente dentro del contenedor:

```
PS C:\Users\Quiryat\Documents\Python\python-mastery-capstone> $response = Invoke-WebRequest -Uri http://localhost:8000/ -UseBasicParsing
PS C:\Users\Quiryat\Documents\Python\python-mastery-capstone> $response.StatusCode
200
PS C:\Users\Quiryat\Documents\Python\python-mastery-capstone> $response.Content
{"message":"Orders API funcionando"}
```